ÉTUDE DE CAS — STRIPE

LIVRABLE 5

Architecture du pipeline de données

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l'infrastructure de données et de calcul pour l'IA

Ce livrable décrit le pipeline qui fait circuler les données entre les différents systèmes de l'architecture : la base OLTP (Aurora PostgreSQL), les bases NoSQL (DocumentDB et Redis) et le data warehouse OLAP (Redshift). Le pipeline combine deux types de traitements : le streaming, pour les données qui doivent être disponibles en temps réel, et le batch, pour les traitements plus lourds qui sont lancés à intervalles réguliers (toutes les heures, chaque semaine ou chaque mois).

1. Principes du pipeline

- **L'OLTP comme source de référence.** Les transactions sont écrites une seule fois dans Aurora PostgreSQL. Toutes les autres bases reçoivent ensuite une copie de ces données, ce qui évite d'avoir des informations différentes d'un système à l'autre.
- **Deux vitesses de traitement.** Le streaming est utilisé quand la donnée doit être disponible en quelques secondes (détection de fraude, stockage des logs). Le batch est utilisé pour les traitements lourds qui n'ont pas besoin d'être faits en continu (chargement du data warehouse, entraînement des modèles).
- **Un traitement incrémental.** Compte tenu des volumes de Stripe, seules les nouvelles données sont traitées à chaque passage. On ne retraite jamais l'ensemble de l'historique.

2. Schéma du pipeline

Le schéma ci-dessous présente les flux de données entre les différents systèmes. Les traits pleins correspondent aux flux en streaming (temps réel) et les traits en pointillés aux traitements batch orchestrés par Airflow. Le schéma complet de l'architecture est présenté dans le livrable 1.

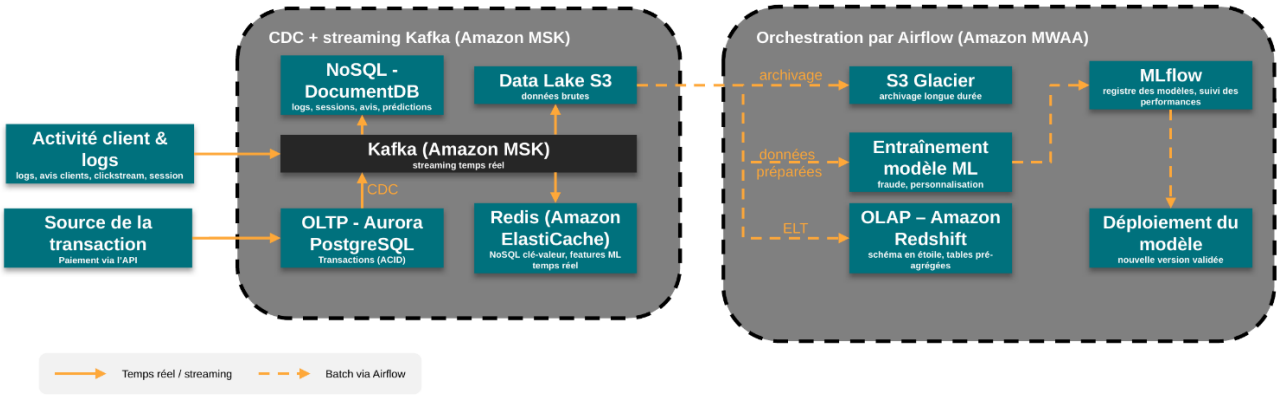


Figure 1 – Flux du pipeline de données (streaming et batch). Le passage de la transaction par le service de détection de fraude avant son enregistrement est présenté au livrable 1.

3. Partie streaming (temps réel)

3.1 Capture des changements de l'OLTP (CDC)

Debezium lit en continu le journal des modifications d'Aurora PostgreSQL. Chaque nouvelle transaction, ou chaque modification (par exemple un paiement qui passe au statut « remboursé »), est publiée dans Kafka (Amazon MSK) en quelques secondes. Cette méthode ne ralentit pas la base OLTP, puisque Debezium lit le journal et n'interroge pas directement les tables.

3.2 Données d'activité client

Les logs, les données de navigation (clickstream, sessions) et les avis clients sont envoyés directement dans Kafka par le site web et les applications mobiles, sans passer par l'OLTP. De la même façon, le service d'inférence publie chaque prédiction du modèle de fraude (score, décision, version du modèle) dans Kafka, sans attendre de réponse : le paiement n'est donc pas ralenti.

3.3 Distribution des données

Kafka organise les données en topics, un par type de données, par exemple « transactions », « sessions », « logs », « avis_clients » et « predictions ». Chaque topic est découpé en partitions, avec l'identifiant du client

comme clé : les événements d'un même client restent dans l'ordre et plusieurs consommateurs peuvent lire en parallèle. Des connecteurs MSK Connect et un service de calcul des features lisent ces topics :

- **Vers DocumentDB :** les logs, les sessions, les avis clients et les prédictions du modèle sont enregistrés sous forme de documents JSON.
- **Vers le Data Lake S3 :** toutes les données sont copiées telles quelles, en format Parquet, et rangées par source, par date et par heure (par exemple raw/transactions/2026/10/09/14/). Ce rangement (partitionnement) permet aux traitements batch de ne lire que les nouveaux fichiers.
- **Vers Redis :** un service de calcul des features, développé en Python et déployé sur Amazon ECS, lit les nouveaux événements et met à jour les features de chaque client et de chaque marchand. Par exemple : le nombre de paiements dans la dernière heure, ou le montant moyen sur 30 jours. Le service d'inférence peut ainsi lire ces features en quelques millisecondes au moment du paiement.

4. Partie batch (orchestrée par Airflow)

Les traitements batch sont planifiés et suivis par Airflow (Amazon MWAA). Chaque traitement correspond à un DAG (Directed Acyclic Graph), c'est-à-dire un enchaînement de tâches avec leur ordre d'exécution.

DAG	Fréquence	Étapes
Chargement du data warehouse	Toutes les heures	1. Copie des nouveaux fichiers (dernière heure) depuis S3 vers des tables temporaires dans Redshift 2. Transformation en SQL : conversion des montants en USD, calcul des segments clients, suppression des doublons 3. Mise à jour des dimensions puis des tables de faits (transactions et journal d'audit) 4. Recalcul des tables pré-agrégées (revenus, fraude, produits) pour les jours concernés, puis rafraîchissement des vues matérialisées
Suivi du modèle	Chaque jour	1. Comparaison de la distribution des features et des scores avec celle des données d'entraînement 2. Calcul de la précision et du recall sur les fraudes confirmées 3. Lancement du DAG d'entraînement si une dérive ou une baisse de performance est détectée (livrable 7)
Entraînement du modèle	Chaque semaine	1. Préparation des données d'entraînement à partir de l'historique du Data Lake 2. Calcul des features avec les mêmes formules que le service de streaming 3. Entraînement du modèle 4. Enregistrement dans MLflow avec ses performances 5. Déploiement uniquement si le nouveau modèle est meilleur que le précédent
Archivage	Chaque mois	Déplacement des fichiers anciens du Data Lake vers S3 Glacier, selon les durées de conservation définies (livrable 6). Retrait de l'OLTP des transactions de plus de 24 mois, déjà présentes dans le Data Lake et Redshift (ces suppressions ne sont pas reportées dans Redshift).

Airflow relance automatiquement une tâche en cas d'échec et envoie une alerte si l'erreur persiste. Une tâche ne démarre que si la précédente a réussi : par exemple, les tables pré-agrégées ne sont recalculées que si la table de faits a bien été mise à jour.

5. Cartographie des flux

Source	Outil	Fréquence	Destination
Aurora PostgreSQL (transactions, clients, marchands…)	Debezium (CDC) + Kafka	Temps réel (quelques secondes)	Kafka
Site web et applications (logs, sessions, avis)	Kafka	Temps réel	Kafka

Source	Outil	Fréquence	Destination
Service d'inférence (prédictions)	Kafka (topic « predictions »)	Temps réel	Kafka
Kafka (Amazon MSK)	MSK Connect (connecteur DocumentDB)	Temps réel	DocumentDB
Kafka (Amazon MSK)	MSK Connect (connecteur S3)	Temps réel	Data Lake S3
Kafka (Amazon MSK)	Service de calcul des features (ECS)	Temps réel	Redis
Data Lake S3	Airflow + SQL Redshift (ELT)	Toutes les heures	Redshift
Data Lake S3	Airflow + MLflow	Hebdomadaire, ou plus tôt en cas de dérive du modèle	Modèle déployé dans le service d'inférence
Data Lake S3	Airflow	Mensuel	S3 Glacier

6. Cohérence et fiabilité des données

- **Pas de doublon à la source.** La clé d'idempotence ('idempotency_key') de la table 'transaction' empêche qu'un même paiement soit enregistré deux fois dans l'OLTP.
- **Pas de perte dans Kafka.** Kafka conserve les messages pendant plusieurs jours et garde en mémoire la position de lecture de chaque consommateur. Si un consommateur tombe en panne, il reprend exactement là où il s'était arrêté.
- **Pas de doublon dans le data warehouse.** Lors du chargement dans Redshift, les lignes sont fusionnées à partir du 'transaction_id'. Si une transaction est chargée deux fois, elle n'apparaît qu'une seule fois dans la table de faits.
- **Cohérence forte et cohérence à terme.** L'OLTP garantit une cohérence forte (propriétés ACID, Atomicité ; Cohérence ; Isolation ; Durabilité). Les autres systèmes sont mis à jour avec un léger décalage : quelques secondes pour DocumentDB, S3 et Redis, et moins d'une heure pour Redshift. Ce décalage est acceptable pour l'analyse et a été défini dans les exigences du livrable 0.
- **Résolution des conflits.** L'OLTP reste la seule source de vérité : les autres systèmes ne modifient jamais une transaction, ils en reçoivent seulement une copie. Si deux versions d'une même ligne arrivent dans le désordre, par exemple après une reprise sur panne, on garde la plus récente grâce à la position dans le journal de l'OLTP, que Debezium ajoute à chaque message. Toutes les copies finissent ainsi par avoir la même valeur que l'OLTP.